

PROJET FINAL - APPLICATION FRONTEND COMPLÈTE

B2 React.js & TypeScript • TDP 16 h • Travail en binôme

Objectif. Concevoir et livrer seuls une application React + TypeScript complète, de la première ligne de code jusqu'à une URL publique accessible depuis un téléphone.

Ce projet évalue l'ensemble du module. Toutes les notions vues depuis la rentrée y sont attendues, et rien de plus : il ne s'agit pas d'apprendre une nouvelle bibliothèque, mais de montrer que vous maîtrisez celles que vous connaissez.

Organisation

Moment	Durée	Ce que vous faites
Lundi 21/09	4 h	Tirage de l'API, cadrage, squelette, routes, premier appel API
Mardi 22/09	4 h	État, formulaires, fonctionnalités
Entre le 22 et le 28	4 h	Travail en autonomie : finalisation, tests, déploiement
Lundi 28/09 matin	1 h 30	Derniers réglages et préparation de la soutenance
Lundi 28/09	2 h 15	Soutenances

Les 4 h en autonomie ne sont pas facultatives : le jalon 3 doit être atteint le 28 au matin.

Constitution des binômes et sujet

- Le travail se fait en binôme, constitué le 21/09 au début de la séance.
- Chaque binôme tire au sort une API publique parmi une liste fournie.
- Deux binômes n'ont pas la même API : le sujet de votre voisin ne vous sera d'aucune aide directe.
- Les APIs sont de difficultés comparables mais présentent chacune une difficulté technique particulière, indiquée dans la fiche des APIs. Cette difficulté fait partie du sujet.
- Aucune API ne demande de clé ni de compte. Toutes ont été vérifiées.

Le thème de l'application découle de l'API tirée : catalogue de Pokémon, galerie d'un musée, bibliothèque, recettes de cuisine, etc.

Périmètre fonctionnel obligatoire

Quelle que soit l'API, l'application doit permettre à un utilisateur de :

- parcourir une liste d'éléments provenant de l'API ;
- rechercher et filtrer cette liste ;
- consulter la fiche détaillée d'un élément, accessible par son URL propre ;
- constituer une sélection personnelle consultable sur une page dédiée ;
- remplir un formulaire validé (contact, proposition d'ajout, signalement — à vous de choisir) ;
- comprendre à tout moment ce que fait l'application : chargement, erreur, résultat vide.

Exigences techniques

Chaque ligne de ce tableau est vérifiée lors de la correction.

#	Exigence	Vu en séance
1	Projet Vite + React + TypeScript, aucune erreur de npm run build	Séance 0
2	Composants typés, props typées, aucun any non justifié	Séance 0
3	Au moins 4 routes dont une route dynamique, plus une page 404	Routage

4	Navigation par Link/NavLink, lien actif visible dans le menu	Routage
5	Données chargées depuis l'API avec useEffect, dépendances correctes	State & asynchrone
6	États loading, error et données traités partout où il y a un appel	State & asynchrone
7	Réponse de l'API typée, y compris les champs pouvant être absents	State & asynchrone
8	Recherche et filtre contrôlés, sans stocker de liste dérivée	Formulaires
9	Un formulaire complet avec validation et message de succès	Formulaires
10	Un état partagé entre plusieurs pages via Context	Formulaires
11	Au moins un hook personnalisé réellement réutilisé	Formulaires
12	Au moins une optimisation justifiée par une mesure avant/après	Performance
13	Au minimum 8 tests, dont une fonction pure et un composant	Tests
14	Application déployée, accessible par une URL publique	Déploiement
15	Rafraîchissement fonctionnel sur une route profonde	Déploiement
16	Aucune adresse d'API en dur, aucun secret dans le dépôt	Déploiement

Ce qui n'est pas demandé

Pour éviter les hors-sujets coûteux :

- aucun backend à développer ;
- aucune authentification ;
- aucune bibliothèque de gestion d'état globale (Redux, Zustand...) : le Context suffit ;
- aucune bibliothèque de formulaires ;
- aucune persistance obligatoire des données ;
- aucune obligation de design élaboré — une interface sobre et lisible suffit.

Une bibliothèque supplémentaire n'est acceptée que si vous pouvez expliquer pourquoi React seul ne suffisait pas. Dans le doute, abstenez-vous.

Les trois jalons

Chaque jalon est vérifié par l'enseignant à la fin de la séance concernée. Un jalon non atteint n'est pas sanctionné en soi, mais il annonce un rendu incomplet.

Jalon 1 — fin de la séance du 21/09

- Le projet démarre avec `npm run dev`.
- Les routes principales existent, même avec des pages presque vides.
- La liste d'éléments s'affiche réellement depuis l'API.
- Le type décrivant la réponse de l'API est écrit.
- Le dépôt Git contient au moins un commit par membre du binôme.

Jalon 2 — fin de la séance du 22/09

- La fiche détail fonctionne par URL et gère l'identifiant inconnu.
- La recherche et le filtre fonctionnent.
- La sélection personnelle fonctionne d'une page à l'autre.
- Le formulaire est en place, avec sa validation.

Jalon 3 — le 28/09 au matin

- Les tests passent avec `npm run test`.
- Une mesure avant/après documente une optimisation.
- L'application est déployée et l'URL fonctionne depuis un téléphone.
- Le README est à jour.

Livrables

Livrable	Forme
Dépôt Git	lien, historique lisible, commits des deux membres
Application déployée	URL publique fonctionnelle
README	à la racine du dépôt, voir le contenu attendu ci-dessous
Journal de décisions	dans le README ou un fichier DECISIONS.md
Checklist de mise en production	la fiche de la séance précédente, complétée

Contenu attendu du README

- le nom des deux membres du binôme et l'API tirée ;
- comment installer et lancer le projet ;
- l'URL de l'application déployée ;
- les fonctionnalités présentes ;
- l'optimisation réalisée, avec la mesure avant et après ;
- ce qui est testé, et ce qui ne l'est pas ;
- ce que vous feriez avec une semaine de plus.

Journal de décisions

Cinq entrées minimum, très courtes, au format :

Décision : la liste des genres n'est pas stockée dans un state.

Pourquoi : elle se calcule à partir des données reçues.

Alternative écartée : un useState mis à jour par un useEffect.

Ce journal pèse autant qu'une fonctionnalité dans l'évaluation : il montre que vous avez choisi, et pas seulement assemblé.

Usage de l'IA

L'IA est autorisée, comme depuis le début du module. La règle ne change pas :

EXPLIQUER -> MODIFIER -> TESTER

- Vous devez pouvoir expliquer n'importe quelle ligne de votre dépôt.
- Vous devez pouvoir la modifier devant l'enseignant.
- Vous devez pouvoir dire pourquoi cette solution plutôt qu'une autre.

En soutenance, chaque binôme reçoit une modification à réaliser en direct, dans son propre code. C'est la partie de l'évaluation qu'aucune génération de code ne remplace.

Un code que vous ne savez pas expliquer sera considéré comme non acquis, qu'il fonctionne ou non. À l'inverse, une fonctionnalité modeste mais parfaitement maîtrisée vaut mieux qu'une application impressionnante que vous ne comprenez pas.

Soutenance — 15 minutes par binôme

Temps	Contenu
5 min	Démonstration de l'application depuis l'URL déployée
4 min	Explication technique de deux points choisis par l'enseignant dans votre code
3 min	Modification demandée, réalisée en direct
3 min	Questions, dont la mesure de performance et vos choix de tests

Les deux membres parlent. La répartition du travail est visible dans l'historique Git et fait partie de l'évaluation.

Barème

Domaine	Points
Socle technique : routage, API, état, formulaires	8
Tests	3
Soutenance : explications, modification en direct	3
Performance mesurée et justifiée	2
Déploiement et mise en production	2
Qualité du code, typage, README et journal de décisions	2

La grille détaillée est distribuée en même temps que ce sujet. Une modulation individuelle de -3 à +3 points peut être appliquée si la soutenance ou l'historique Git révèle un déséquilibre important entre les deux membres.